The services also expose metrics in the Prometheus exposition format (e.g., looking at the `front-end` service we forwarded to the local machine earlier). The following output is edited to fit:

```
$ curl localhost:8888/metrics
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 156.37216999999978
...
```

In this case, the source is the `front-end` service, and the signal type is metrics.

> **Note**
>
> When talking with customers, I notice that many are facing the same challenge: feeling the need to collect and retain all metrics because "we might need them." Try to prune as much as possible on ingest and check if metrics that land in destinations are actually used. These are some questions you can ask yourself about whether collecting a particular metric is necessary:
>
> - Is that metric used in an alert? Is it used in a dashboard?
> - Can you think of a use case where you could derive insights from the metric? Would an aggregate be sufficient?
> - Would a trace be a better choice here?

To make the metrics actionable, for an SRE to assess indicators, for example, we would need the following:

- An agent (e.g., Prometheus or OpenTelemetry collector) to scrape the metrics
- A destination, such as Prometheus or, for long-term storage and federation, Cortex or Thanos

For now, we will not consider traces, since they require a little bit more setup (both from the telemetry side and the destination), but in a nutshell, you can think of traces as providing an overview of the execution across services, including the duration and status of the service that successfully handled the request.

We've had an initial exposure to logs and metrics now. Before we go deeper into cloud observability throughout the rest of this book, let's finish this chapter by looking at the challenges you will need to keep in mind when dealing with cloud-native systems and how observability can help to address them.

1.5 Challenges and how observability helps

Cloud-native environments and the apps running on them face several challenges. In this subsection, we discuss the most pressing of these and show how observability can help.